Data Structure 3

Paul Blondel

CNAM

September 1st, 2018

'I will, in fact, claim that the difference between a bad programmer and a good one is whether he considers his code or his data structures more important. Bad programmers worry about the code. Good programmers worry about data structures and their relationships.'

Linus Torvalds

Recall

Sorted arrays are **more efficient in search**

Besides, many algorithms need sorted arrays to work:

- Dijkstra's algorithm (finding shortest path)
- Prim's algorithm (finding minimum spanning tree)
- Kurskal's algorithm (finding minimum spanning tree)
- many more...

We will study the following sorting approaches:

- Selection sort
- Insertion sort
- Quick sort
- Heap sort
- Bubble sort

Principle

Swap the current value with the **smallest unsorted value**

```
i=0;j=0;
while i < Array.length − 1 do
    min=i;
    j=i+1;
    while j < Array.length do
        if Array[j] < Array[min] then
            min=j;
        end
        j=j+1;
    end
    temp=Array[i];
    Array[i]=Array[min];
    Array[min]=temp;
    i=i+1;
end
```

Complexities:

comparisons $= O(N^2)$

swaps $= O(N)$

Example:

step 0: **29**, 64, 73, 34, 20

step 1: *20*, **64**, 73, 34, 29

step 2: *20*, *29*, **73**, 34, 64

step 3: *20*, *29*, *34*, **73**, 64

step 4: *20*, *29*, *34*, *64*, *73*

Principle

Move from **right to left** the unsorted value **to the best position**

```
i=1;j=0;
while i < Array.length do
    val = Array[i];
    j=i;
    while j > 0 and Array[j − 1] > val
     do
        Array[j] = Array[j-1];
        j=j-1;
    end
    Array[j] = val;
end
```

Example:

step 0: *29*, **20**, 73, 34, 64
step 1: *20*, *29*, **73**, 34, 64
step 2: *20*, *29*, *73*, **34**, 64
step 3: *20*, *29*, *34*, *73*, **64**
step 4: *20*, *29*, *34*, *64*, *73*

Complexities:

- Best: $O(N)$ for comparisons, $O(1)$ for swaps
- Worst/Average: $O(N^2)$ for comparisons and swaps

Principle

Compare each value to the next and swap if required (bubbling)

```
i=Array.length-1;j=0;
while i >= 0 do
    j=1;
    while i <= i do
        if Array[j-1] > Array[j] then
            temp = Array[j-1];
            Array[j-1] = Array[j];
            Array[j] = temp;
        end
        j=j+1;
    end
    i=i-1;
end
```

Example:

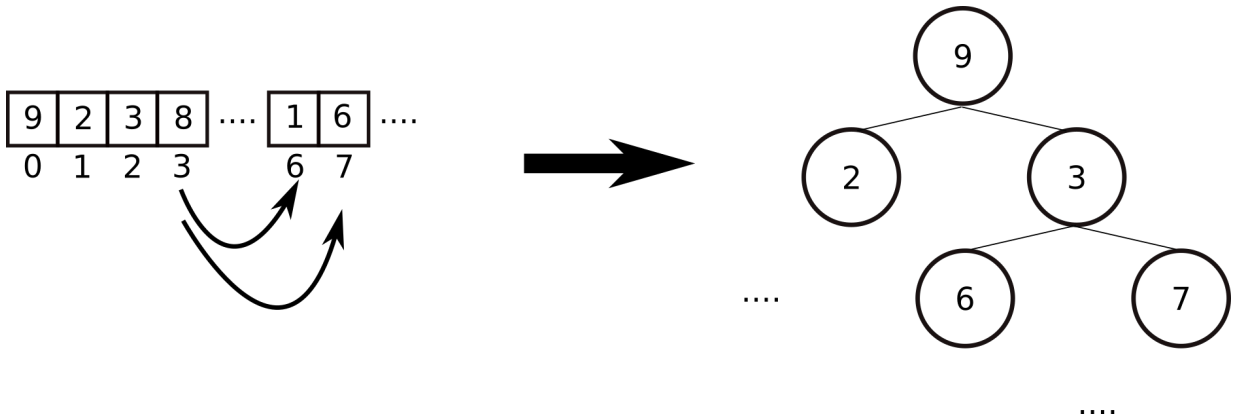
step 0: **7**, 5, 2, 4, 3, 9
step 1: *5*, **7**, 2, 4, 3, 9
step 2: *5*, *2*, **7**, 4, 3, 9
step 3: *5*, *2*, *4*, **7**, 3, 9
step 4: *5*, *2*, *4*, *3*, **7**, 9
step 5: *5*, *2*, *4*, *3*, *7*, *9*

Complexities:

- Best: $O(N)$ for comparisons, $O(1)$ for swaps
- Worst/Average: $O(N^2)$ for comparisons, $O(N^2)$ for swaps

Principle

- The input array is **seen as a *binary tree*** where:
  - element i has children at index $2 \times i$ and $2 \times i + 1$
- The binary tree is *then* **transformed into a *binary heap***
- The **root node value is removed from the tree (it's sorted now)**[1]
- The new tree is **transformed back to a binary heap**, and so on

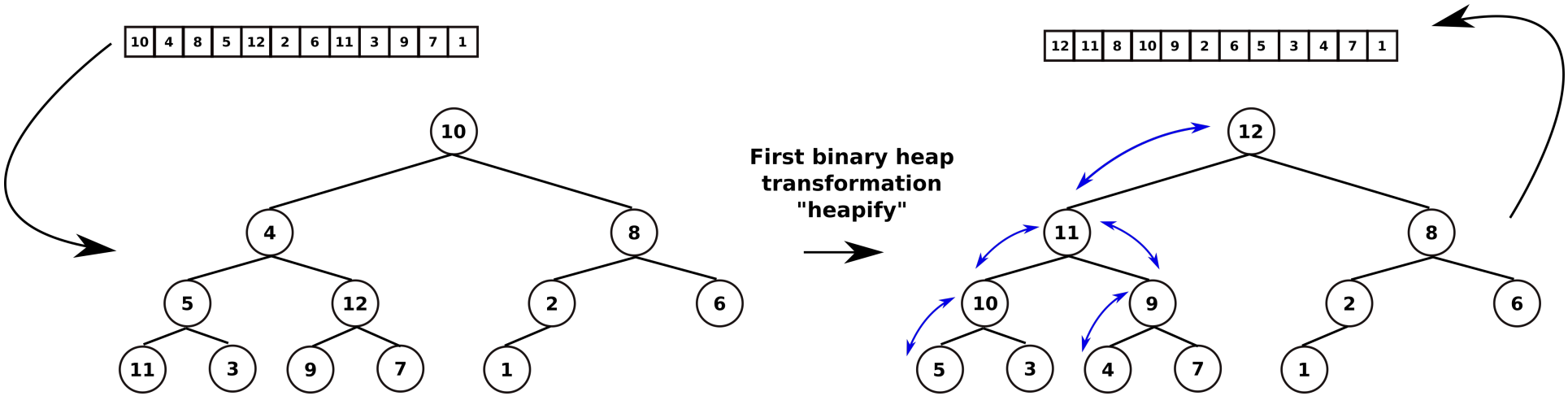


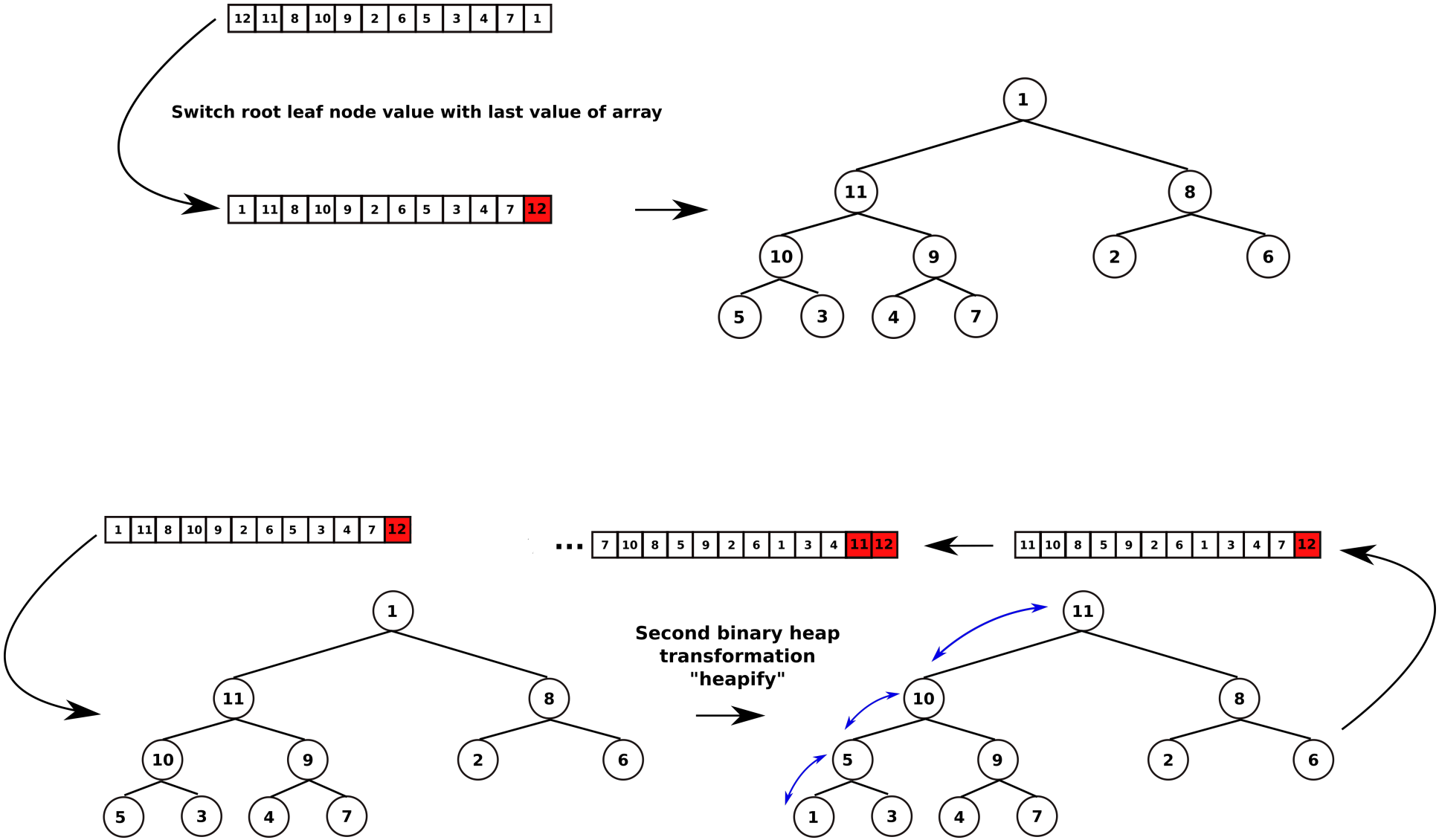
[1] The previous root node value is **not reinserted** in the tree.

A *binary tree* is transformed into a *(max) binary heap* if:

- each node has a **value greater than those of its two children**

Example:




12 11 8 10 9 2 6 5 3 4 7 1
Switch root leaf node value with last value of array
1 11 8 10 9 2 6 5 3 4 7 12
1
11
8
10
9
2
6
5
3
4
7
1 11 8 10 9 2 6 5 3 4 7 12
...
7 10 8 5 9 2 6 1 3 4 11 12
11 10 8 5 9 2 6 1 3 4 7 12
Second binary heap
transformation
"heapify"
1
11
8
10
9
2
6
5
3
4
7
11
10
8
5
9
2
6
1
3
4
7

```
Function build_heap(Array) is
    i=N/2;
    while i >= 1 do
        heapify(Array,i);
        i=i-1;
    end
end
```

```
Function heapify(Array, i) is
    left=2 × i;
    right=2 × i + 1;
    if (left <= N)and(Array[left] > A[i]) then
        max=left;
    else
        max=i;
    end
    if (right <= N)and(Array[right] > A[max]) then
        max=right;
    end
    if max! = i then
        temp = Array[i];
        Array[i] = Array[max];
        Array[max] = temp;
        heapify(A, max);
    end
end
```

```
//Heap-sorting code
build_heap(Array);
i=N;
while i >= 1 do
    temp = A[1];
    A[1] = A[i];
    N=N-1;
    heapify(A, 1);
    j=j-1;
end
```

Complexities: $O(N \times log(N))$

Principle

- Based on the **divide and conquer** approach
- Pick a ***pivot***[2] **element for partitioning** (first = last element)[3]
- For each *pivot*: *partitioning* = reordering the array:
  - elements with **lesser values must be at the left of the pivot**
  - elements with **bigger values must be at the right of the pivot**
  - we **change the position** of the pivot accordingly
  - when **done = we change the pivot**
    1. new pivot = **element at the left of the previous pivot element**
    2. in this new partition[4], we rearrange elements as explained above
    3. we repeat (1) and (2) until reaching the extreme left
    4. we **do the same in the other direction** (new pivots will be on the right)

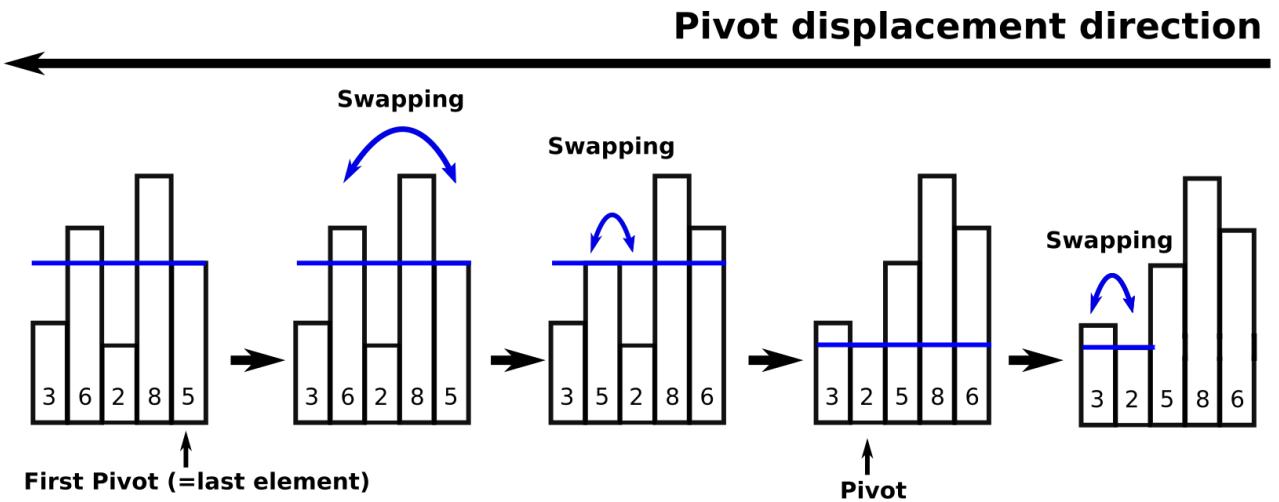
Note

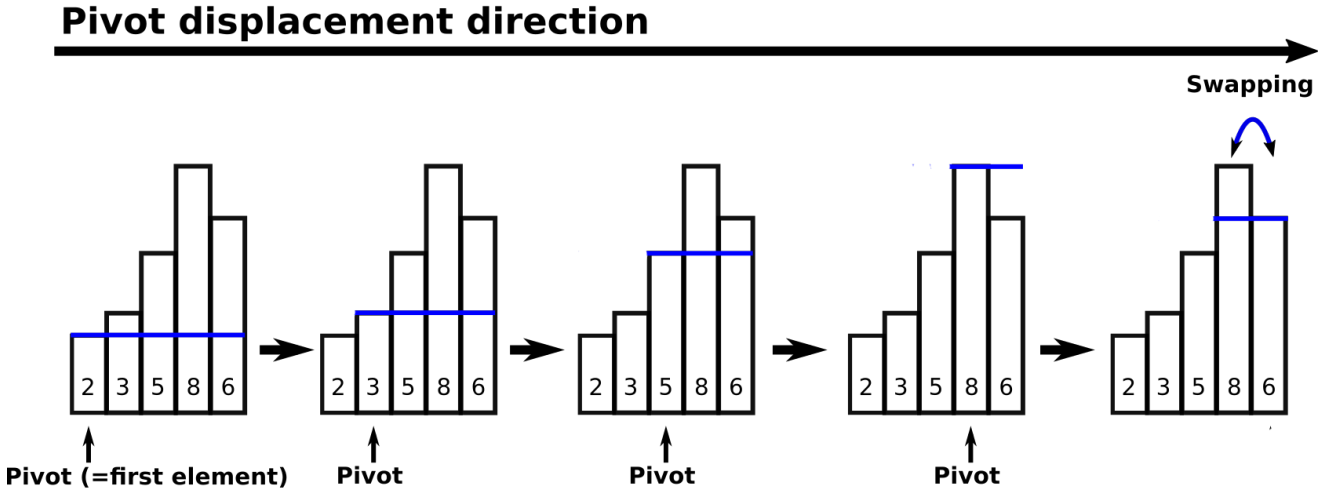
A practice: one of the fastest sorting algorithm

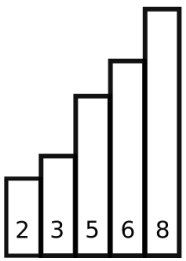
[2] Pivot, like: *pivoting*, or *rotating* around something

[3] Note that other approaches exist: taking the median element value, random, etc.

[4] Partition: subpart of the array going from the pivot to the extreme left (or right)


Pivot displacement direction
Swapping
Swapping
Swapping
3 6 2 8 5
3 6 2 8 5
3 5 2 8 6
3 2 5 8 6
3 2 5 8 6
First Pivot (=last element)
Pivot


Pivot displacement direction
Swapping
2 3 5 8 6
2 3 5 8 6
2 3 5 8 6
2 3 5 8 6
2 3 5 8 6
Pivot (=first element)
Pivot
Pivot
Pivot


2 3 5 6 8

```
Function partition(Array, low, high) is
    pivot=Array[high];
    i=low-1;
    j=low;
    while k <= high-1 do
        if Array[j] < pivot then
            i=i+1;
            temp=Array[i];
            Array[i]=Array[j];
            Array[j]=temp;
        end
        temp=Array[i+1];
        Array[i+1]=Array[high];
        Array[high]=temp;
    end
    return i+1;
end

Function quicksort(Array, low, high) is
    if low < high then
        p = partition(Array, low, high);
        quicksort(Array, low, p-1);
        quicksort(Array, p+1, high);
    end
end

//Quick sorting part
quicksort(A, 0, N -1);
```

Complexities:

- at best: $O(N \times log(N))$
- at worst: $O(N^2)$
- in average: $O(N \times log(N))$

- Selection sort:
  - performs **few writes/swaps** in average (good for embedded systems)
- Selection and insertion sort:
  - fast approach for **small arrays**
- Bubble: theoretically and in practice: **not a good solution**
- Heapsort:
  - best theoretical approach (**best time complexities**)
  - more suitable for **average to big arrays**
- Quicksort:
  - **not as good as heapsort theoretically**
  - in practice: **fastest sorting approach**
  - not fast if input array already sorted